CSE234 - STL Algorithm

The `algorithm` header provides a set of common computer science algorithms that operate primarily on containers (`std::list` and `std::vector` are examples of containers). To use it, `#include <algorithm>` . All functions provided in `algorithm` are in the `std` namespace. All algorithms have the option to provide an "execution policy", but these will not be discussed in this course.

For this class, we'll use `std::transform()` as an introduction to the STL algorithms. All the algorithms follow a similar pattern of usage. But first, we'll discuss the mechanisms to provide callbacks: function pointers, functors (also called function objects), and lambdas. C++ treats functions as "first-class citizens", which basically means functions can be stored in variables, used as parameters to other functions, and can be returned from other functions. This directly applies to STL algorithms, since many of them expects a callback function as one of their parameters.

Function Pointers, Functors, and Lambda Expressions

Before discussing `std::transform()` , we'll describe the mechanisms for passing callback functions.

Function Pointers

To declare a function pointer, you need to provide the return type, name of the function pointer variable (which has the pointer asterisk in front of it and is wrapped in parentheses), and the list of types expected for the parentheses. In the following example, the third parameter of

`addOrSubtract()` is a function pointer that returns an integer, and expects two integers as its parameters. The name of the function pointer is `operation`. It can be used directly as a function, as seen in the return statement of `addOrSubtract()`.

```
#include <assert.h>
// addOrSubtract takes two integers, x and y, that should be
// added or subtracted, and a function pointer, operation
// that performs the operation
int addOrSubtract(int x, int y, int (*operation)(int, int)) {
    return operation(x, y);
}

int add(int x, int y) {
    return x + y;
}

int subtract(int x, int y) {
    return x - y;
}

int main() {
    // to use add, we need to use an ampersand
    // on add's name to pass its address
    int additionResult = addOrSubtract(2, 2, &add);
    assert(additionResult == 4);
    // same for subtract, we need the ampersand to pass its address
    int subtractionResult = addOrSubtract(3, 2, subtract);
    assert(subtractionResult == 1);
}
```

C++ 11 added a header `<functional>` that includes the `std::function` type. Below, the `addOrSubtract` function has been modified to use `std::function` as its type:

```
// need to include the functional header
#include <functional>
// std::function is a generic type
// the template argument is the return type, followed by the parameter
// list's data types in parentheses
// so, operation is a function that returns an int
```

```
//  and expects two ints as parameters
int addOrSubtract(int x, int y, std::function<int(int,int)> operation)
    return operation(x, y);
}
// the rest of the program would be the same
```

Functors

A functor is a class/struct that overloads the function call operator, i.e., it implements `operator ()`. Typically, functors only implement this operator, although they can implement other functions as well. The advantage of functors is that they can maintain state via their member variables. The below example implements a functor to computer the average of values in an array:

```
#include <iostream>
#include <array>

// Average is a functor since it implements the call operator
template <typename T>
struct Average {
    // Constructor to init fields to 0
    Average() {
        sum = 0;
        count = 0;
    }

    // operator() adds the value passed to it to the sum,
    // and increments the count
    void operator()(T value) {
        sum += value;
        ++count;
    }

    // property to return computed average
    double average() const {
        if (count == 0) return 0;
        else return static_cast<double>(sum) / count;
    }
private:
    T sum;
    unsigned count;
```

```
};

int main() {
    std::array<int, 7> values = { 2, 6, 24, 76, 3, 67, 5 };
    // IMPORTANT: declare an instance of functor
    Average<int> average;

    for (int x: values) {
        // overloading operator() allows us to use the object
        // like it was a function by calling it
        average(x);
    }

    std::cout << "Average = " << average.average() << std::endl;

    return 0;
}
```

Lambda Expressions

Also called "lambdas" or "anonymous functions", lambda expressions allow you to put a function anywhere you would put an expression, such as the right-hand side of an equals sign or as an argument to a function. Lambdas consist of three parts: a capture, a parameter list, and a body. The capture is denoted with square brackets, and allows you to pass local variables to the lambda's scope. If the lambda needs to modify a variable it captures, the variable must be passed by reference using the ampersand (&). The parameter list is like a typical parameter list for a normal function. It has the types and names of the expected arguments to the lambda. Finally, the body contains the expressions that will be executed when the lambda is called. The body has its own local scope like any other block would. Remember that the body of the lambda cannot access any local variables from the block it is called in unless they are passed via the capture. Below is an example of using a lambda to compute the average of the same array from the functor example:

```
#include <iostream>
#include <array>
```

```
int main() {
    std::array<int, 7> values = { 2, 6, 24, 76, 3, 67, 5 };

    // we need to declare variables for the sum and count locally now
    int count = 0;
    int sum = 0;

    // we can either use auto type or std::function<> to declare a
    // lambda "variable"
    // to allow the lambda to modify the values of count and sum,
    // they must be passed by reference
    auto average = [&count, &sum] (int value) mutable {
        sum += value;
        ++count;
    }; // since this is an expression we need a semicolon at the end

    for (int x: values) {
        average(x);
    }

    std::cout << "Average = " << static_cast<double>(sum) / count << s

    return 0;
}
```

Transform

`std::transform()` has two variants. The first takes the start and end iterator of a collection, an iterator for where to start storing the results of the transform, and a function callback to apply to each element of the collection. The callback should return the modified element. If you want to apply `std::transform()` to a whole collection (i.e., list or vector), simply pass the iterators returned by `begin()` and `end()` to the function. However, you can also pass other iterators if you only want to transform a subset of the collection (of course, the second iterator must come after the first). Below is an example that converts a single string to all uppercase. There are three versions of the example, demonstrating how to use each of the callback mechanisms.

Example (lambda):

```cpp
#include <algorithm>
#include <string>

int main() {
    std::string str = "hello world";
    std::transform(str.begin(), str.end(), str.begin(), [](const char&
        return toupper(c);
    });
    // str == "HELLO WORLD"
    return 0;
}
```

Example (functor):

```cpp
#include <algorithm>
#include <string>

struct ToUpper {
    char operator()(char c) {
        return toupper(c);
    }
};

int main() {
    std::string str = "hello world";
    // since this functor does not have state,
    // pass it a call to its constructor directly
    std::transform(str.begin(), str.end(), str.begin(), ToUpper());
    // str == "HELLO WORLD"
    return 0;
}
```

Example (function pointer):

```cpp
#include <algorithm>
#include <string>

// we could have used toupper directly in std::transform
char toUpper(char c) {
    return toupper(c);
}

int main() {
    std::string str = "hello world";
    // need the & to pass address of the function
    std::transform(str.begin(), str.end(), str.begin(), &toUpper());
    // str == "HELLO WORLD"
    return 0;
}
```

For the next version of `std::transform()`, I'll provide an example first and then explain:

```cpp
#include <algorithm>
#include <cmath>
#include <vector>

// this will calculate the hypotenuses of triangles and store them in
// third vector using std::transform
int main() {
    // vector of the bases
    std::vector<int> bases = { 1, 2, 3, 4, 5, 6, 7, 8 };
    // vector of the heights
    std::vector<int> heights = { 2, 3, 4, 5, 12, 13, 24, 15 };
    // vector to store the hypotenuses in
    std::vector<double> hypotenuses(bases.size());
    // note that each parameter has a line comment above it
    //  explaining its purpose
    std::transform(
        // the start iterator of the first range
        bases.begin(),
        // the end iterator of the first range
        bases.end(),
        // the start iterator of the second range
        // the second range is assumed to be at least
        //  as large as the first
        heights.begin(),
```

```
        // the start iterator of the destination range
        hypotenuses.begin(),
        // the callback returns the data type of the destination
        // and takes a parameter from each of the input ranges
        // a will be an element of the first range,
        // b is from the second
        [](int a, int b) {
            // a^2 + b^2 = c^2 => c = sqrt(a^2 + b^2)
            // square root is the same as the 0.5 power
            // return value is stored in the destination range
            return pow(pow(a, 2) + pow(b, 2), 0.5);
        }
    );
    /*
        hypotenuses == {
                        2.23607,
                        3.60555,
                        5,
                        6.40312,
                        13,
                        14.3178,
                        25,
                        17
                       }
    */
    return 0;
}
```

This version of transform is basically used to combine elements of two ranges. If you view the example on C++ Reference (https://en.cppreference.com/w/cpp/algorithm/transform.html) you can see that this could be used to edit a single collection as well. The first two parameters are the start and end of the first range. The third parameter is the start of the second range (the end is deduced based on the distance between the start and end of the first range). Of course, the second range would need to be at least as large as the first. The fourth parameter is the start iterator of the destination range. The last parameter is a callback that has two parameters, one from each input range. Its return value will be stored in the destination range.